Sequential Search, Brute Force String Matching

Sequential Search

Question 1

Write a program to implement Sequential Search to find a given element in an array. Input:

Array: 12, 25, 8, 45, 32, 19, 50

Key: 32

Output:

Element found at position 5

Number of comparisons = 5

main.py | Share | Run | Output | Clear

```
# Sequential Search Program

arr = [12, 25, 8, 45, 32, 19, 50]
key = 32

comparisons = 0

for i in range(len(arr)):
    comparisons += 1
    if arr[i] == key:
        print("Element found at position", i + 1)
        print("Number of comparisons =", comparisons)
        break
else:
    print("Element not found")
    print("Number of comparisons =", comparisons)
```

```
Element found at position 5
Number of comparisons = 5

=== Code Execution Successful ===
```

Question 2

Implement Sequential Search for an unsuccessful search.

Input:

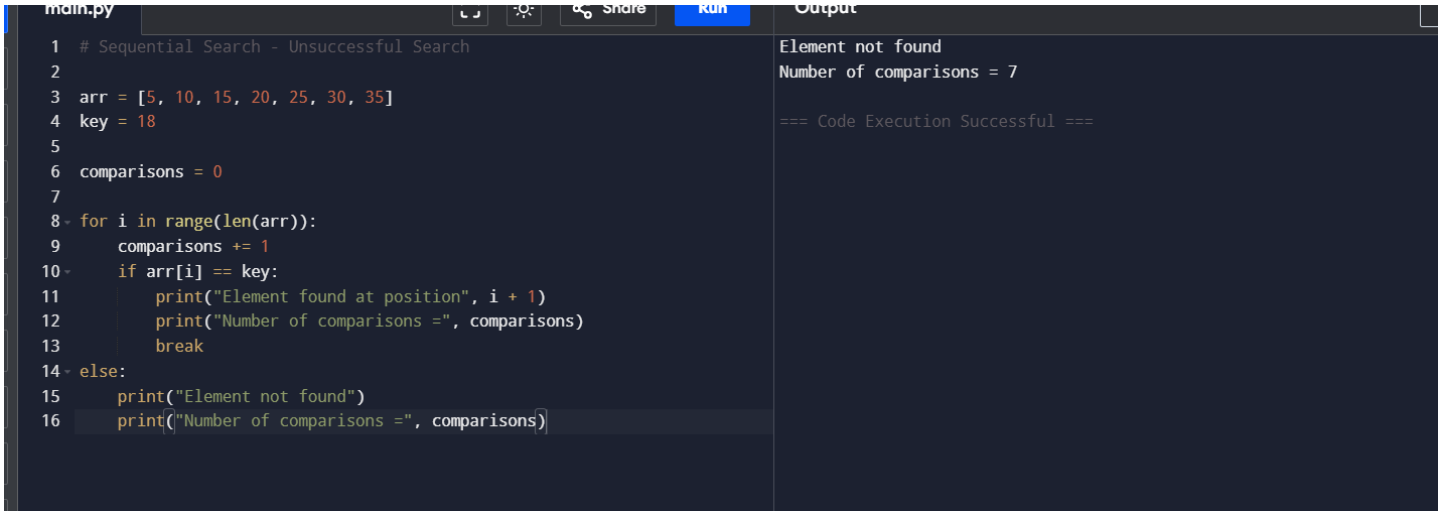
Array: 5, 10, 15, 20, 25, 30, 35

Key: 18

Output:

Element not found

Number of comparisons = 7



Question 3

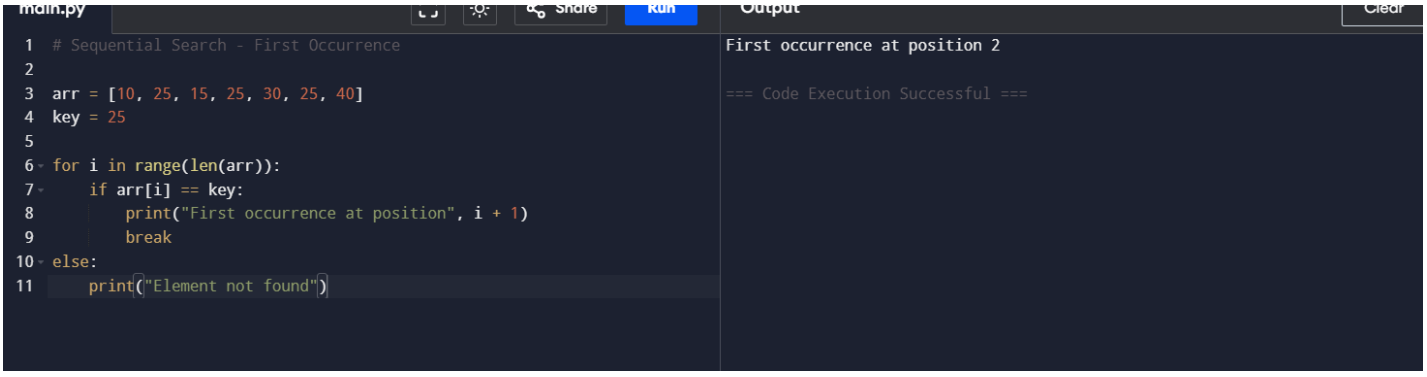
Write a program to find the first occurrence of an element using Sequential Search.

Input:

Array: 10, 25, 15, 25, 30, 25, 40
Key: 25

Output:

First occurrence at position 2



Question 4

Implement Sequential Search to find all occurrences of a given element.

Input:

Array: 7, 12, 7, 25, 18, 7, 30, 7

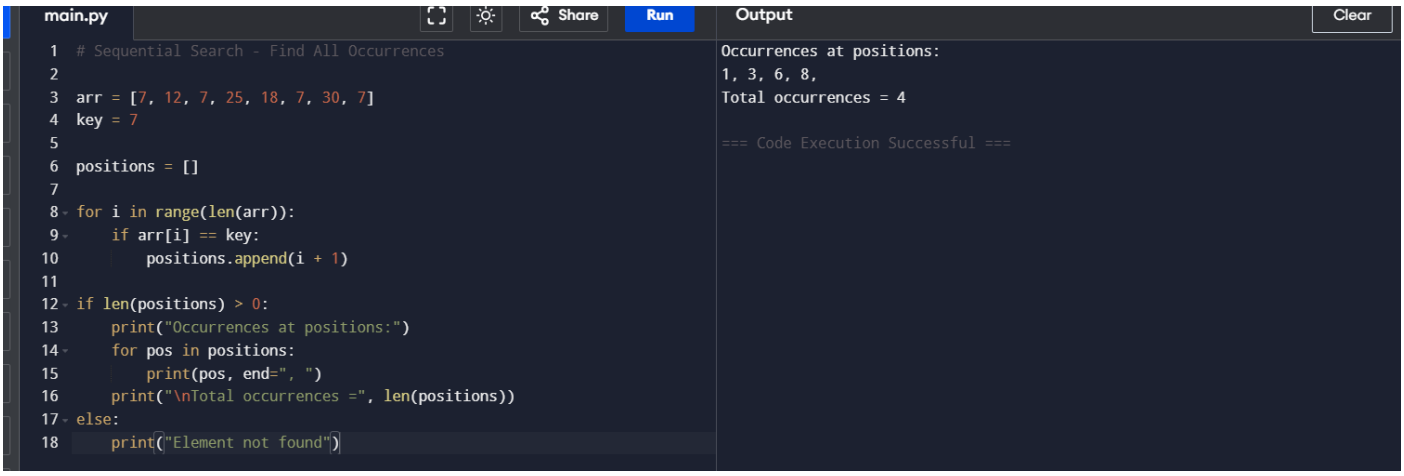
Key: 7

Output:

Occurrences at positions:

1, 3, 6, 8

Total occurrences = 4



Question 5

Write a program to count:

- Total comparisons
- Total matches
- Total mismatches

during Sequential Search.

Input:

Array: 3, 6, 9, 12, 15, 18, 21

Key: 15

```python
# Sequential Search - Count Comparisons, Matches, and Mismatches

arr = [3, 6, 9, 12, 15, 18, 21]
key = 15

comparisons = 0
matches = 0
mismatches = 0

for i in range(len(arr)):
    comparisons += 1

    if arr[i] == key:
        matches += 1
        break
    else:
        mismatches += 1

print("Total comparisons =", comparisons)
print("Total matches =", matches)
print("Total mismatches =", mismatches)
```

```
Occurrences at positions:
1, 3, 6, 8,
Total occurrences = 4

=== Code Execution Successful ===
```

Question 6

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:

Array: 14, 9, 22, 35, 18, 41, 27
Key: 18

Output:

Position found

Comparison count

```
print("Ordinary Sequential Search:")
if position1 != -1:
    print("Position found:", position1)
else:
    print("Element not found")
print("Comparison count:", comparisons1)


# Sentinel Sequential Search
arr2 = arr.copy()
arr2.append(key)   # Add key as sentinel

comparisons2 = 0
i = 0

while True:
    comparisons2 += 1
    if arr2[i] == key:
        break
    i += 1

if i < len(arr2) - 1:
    print("\nSentinel Sequential Search:")
    print("Position found:", i + 1)
else:
    print("\nElement not found")

print("Comparison count:", comparisons2)
```

```
Ordinary Sequential Search:
Position found: 5
Comparison count: 5

Sentinel Sequential Search:
Position found: 5
Comparison count: 5

=== Code Execution Successful ===
```

Question 7

Write a program to perform Sequential Search on an array of student register numbers.

Input:

Register Numbers:

101, 102, 103, 104, 105, 106

Search Key: 104

Output:

Register Number found at position 4

```
# Sequential Search - Student Register Numbers

register_numbers = [101, 102, 103, 104, 105, 106]
key = 104

for i in range(len(register_numbers)):
    if register_numbers[i] == key:
        print("Register Number found at position", i + 1)
        break
else:
    print("Register Number not found")
```

```
Register Number found at position 4

=== Code Execution Successful ===
```

Question 8

Implement Sequential Search for string data.

Input:

Names:

Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:

Deepa

Output:

Name found at position 4

```python
# Sequential Search - String Data

names = ["Anu", "Bala", "Charan", "Deepa", "Esha", "Farhan"]
key = "Deepa"

for i in range(len(names)):
    if names[i] == key:
        print("Name found at position", i + 1)
        break
else:
    print("Name not found")
```

```
Name found at position 4

=== Code Execution Successful ===
```

Question 9

Write a program to perform Sequential Search on a 2-D matrix.
Input:

12 8 15

5 18 27

9 11 24

Key = 24

Output:

Element found at Row 3 Column 3

```python
# Sequential Search on a 2-D Matrix

matrix = [
    [12, 8, 15],
    [5, 18, 27],
    [9, 11, 24]
]

key = 24
found = False
comparisons = 0

# Sequential Search
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        comparisons += 1

        if matrix[i][j] == key:
            print("Element found at position Row", i + 1, "Column", j + 1)
            print("Number of comparisons =", comparisons)
            found = True
            break

    if found:
        break

if not found:
    print("Element not found")
```

```
Element found at position Row 3 Column 3
Number of comparisons = 9

=== Code Execution Successful ===
```

Question 10

Implement Sequential Search and analyze its performance.

Input:

Array:

45, 23, 67, 12, 89, 34, 56, 78, 90, 11,

29, 73, 18, 64, 37

Search Keys:

73

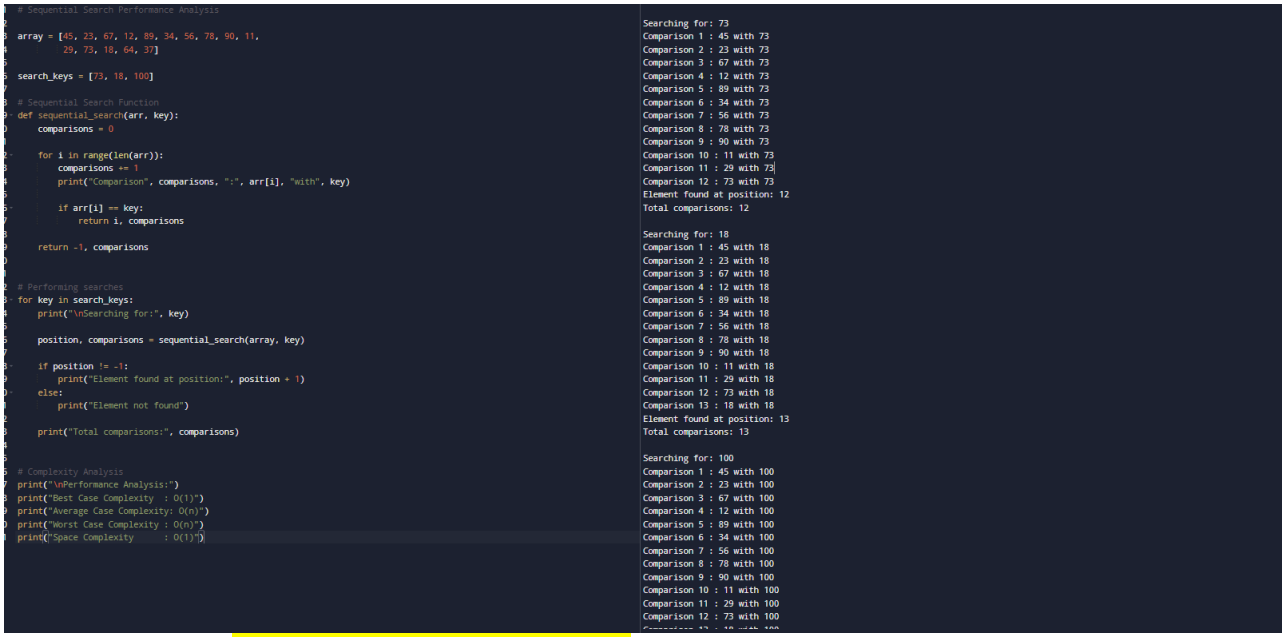
18

100

Tasks:

1. Show each comparison.

2. Count comparisons for every search.
3. Identify best, average, and worst cases.
4. Determine:
    - Best-case complexity
    - Average-case complexity
    - Worst-case complexity
    - Space complexity



```python
# Sequential Search Performance Analysis

array = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11,
         29, 73, 18, 64, 37]

search_keys = [73, 18, 100]

# Sequential Search Function
def sequential_search(arr, key):
    comparisons = 0

    for i in range(len(arr)):
        comparisons += 1
        print("Comparison", comparisons, ":", arr[i], "with", key)

        if arr[i] == key:
            return i, comparisons

    return -1, comparisons


# Performing searches
for key in search_keys:
    print("\nSearching for:", key)

    position, comparisons = sequential_search(array, key)

    if position != -1:
        print("Element found at position:", position + 1)
    else:
        print("Element not found")

    print("Total comparisons:", comparisons)


# Complexity Analysis
print("\nPerformance Analysis:")
print("Best Case Complexity  : O(1)")
print("Average Case Complexity: O(n)")
print("Worst Case Complexity : O(n)")
print("Space Complexity      : O(1)")
```

```
Searching for: 73
Comparison 1 : 45 with 73
Comparison 2 : 23 with 73
Comparison 3 : 67 with 73
Comparison 4 : 12 with 73
Comparison 5 : 89 with 73
Comparison 6 : 34 with 73
Comparison 7 : 56 with 73
Comparison 8 : 78 with 73
Comparison 9 : 90 with 73
Comparison 10 : 11 with 73
Comparison 11 : 29 with 73
Comparison 12 : 73 with 73
Element found at position: 12
Total comparisons: 12

Searching for: 18
Comparison 1 : 45 with 18
Comparison 2 : 23 with 18
Comparison 3 : 67 with 18
Comparison 4 : 12 with 18
Comparison 5 : 89 with 18
Comparison 6 : 34 with 18
Comparison 7 : 56 with 18
Comparison 8 : 78 with 18
Comparison 9 : 90 with 18
Comparison 10 : 11 with 18
Comparison 11 : 29 with 18
Comparison 12 : 73 with 18
Comparison 13 : 18 with 18
Element found at position: 13
Total comparisons: 13

Searching for: 100
Comparison 1 : 45 with 100
Comparison 2 : 23 with 100
Comparison 3 : 67 with 100
Comparison 4 : 12 with 100
Comparison 5 : 89 with 100
Comparison 6 : 34 with 100
Comparison 7 : 56 with 100
Comparison 8 : 78 with 100
Comparison 9 : 90 with 100
Comparison 10 : 11 with 100
Comparison 11 : 29 with 100
Comparison 12 : 73 with 100
```

Brute Force String Matching

Question 1

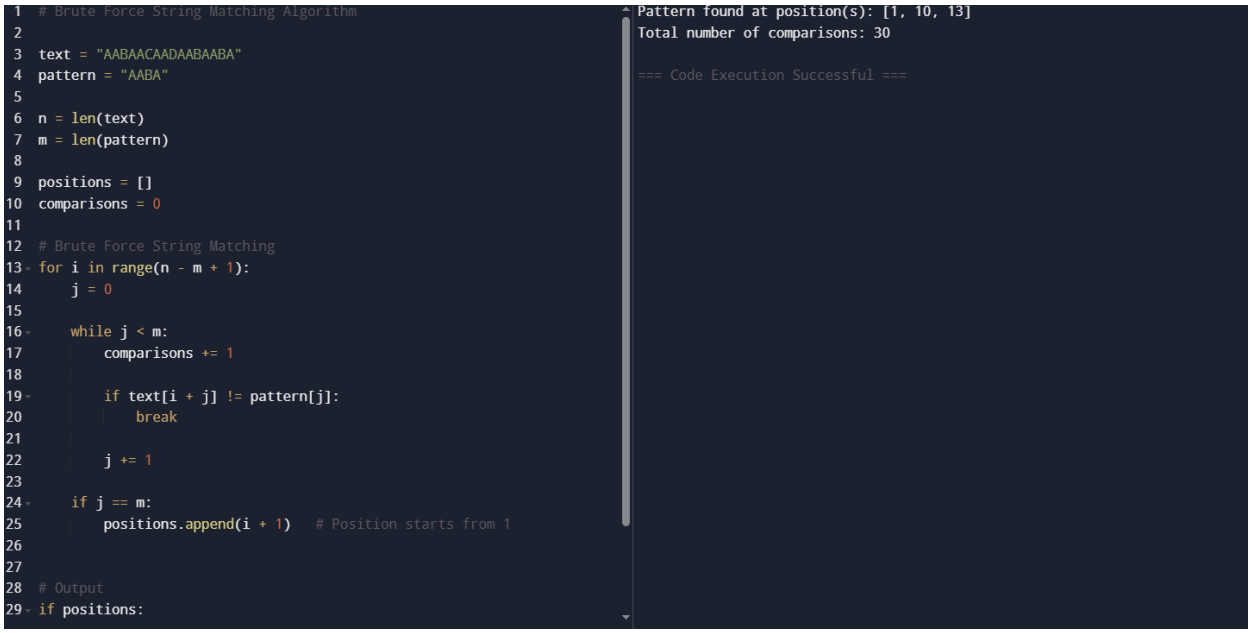
Implement the Brute Force String Matching Algorithm to search for a pattern in a given text.

Input:

- Text = "AABAACAADAABAABA"
- Pattern = "AABA"

Output:

- Position(s) where the pattern occurs.
- Total number of comparisons.



```python
# Brute Force String Matching Algorithm

text = "AABAACAADAABAABA"
pattern = "AABA"

n = len(text)
m = len(pattern)

positions = []
comparisons = 0

# Brute Force String Matching
for i in range(n - m + 1):
    j = 0

    while j < m:
        comparisons += 1

        if text[i + j] != pattern[j]:
            break

        j += 1

    if j == m:
        positions.append(i + 1)   # Position starts from 1


# Output
if positions:
```

```
Pattern found at position(s): [1, 10, 13]
Total number of comparisons: 30

=== Code Execution Successful ===
```

Question 2

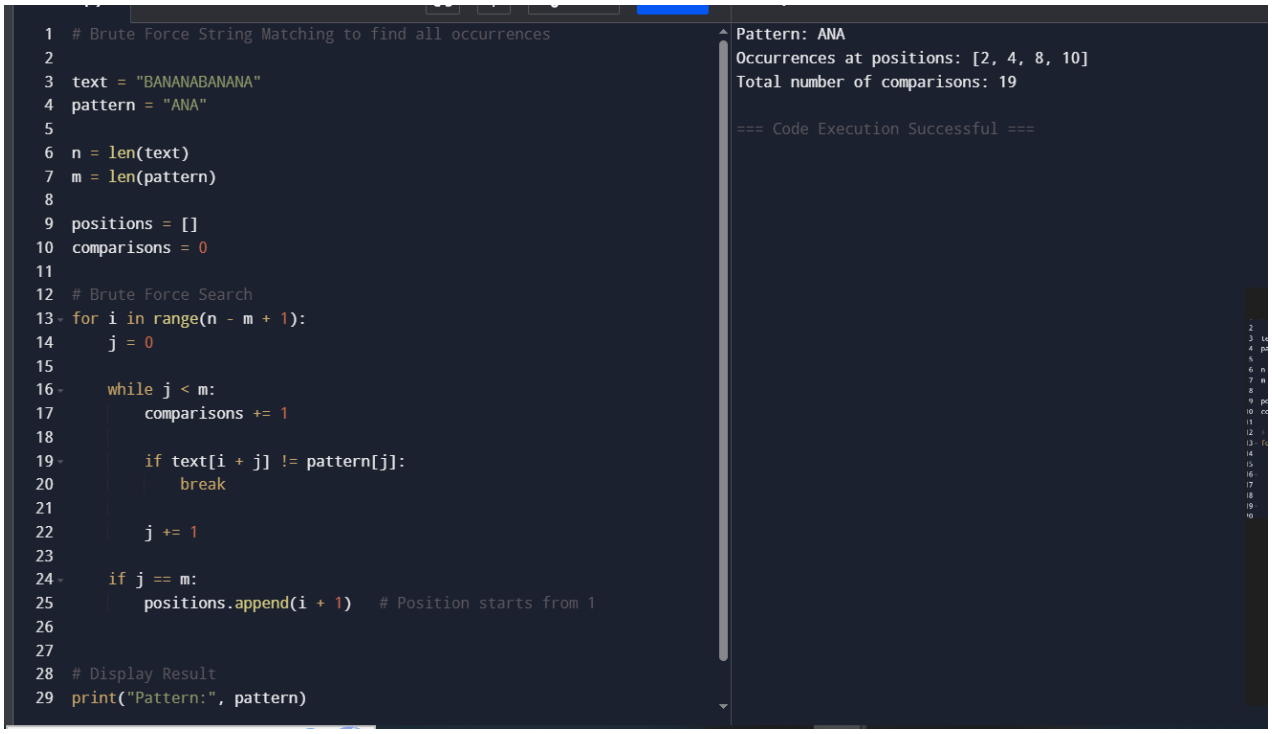
Write a program to find all occurrences of a pattern in a text using Brute Force String Matching.

Input:

- Text = "BANANABANANA"
- Pattern = "ANA"

Output:

- Occurrences at positions 1, 3, 7, 9.



```python
# Brute Force String Matching to find all occurrences

text = "BANANABANANA"
pattern = "ANA"

n = len(text)
m = len(pattern)

positions = []
comparisons = 0

# Brute Force Search
for i in range(n - m + 1):
    j = 0

    while j < m:
        comparisons += 1

        if text[i + j] != pattern[j]:
            break

        j += 1

    if j == m:
        positions.append(i + 1)   # Position starts from 1


# Display Result
print("Pattern:", pattern)
```

```
Pattern: ANA
Occurrences at positions: [2, 4, 8, 10]
Total number of comparisons: 19

=== Code Execution Successful ===
```

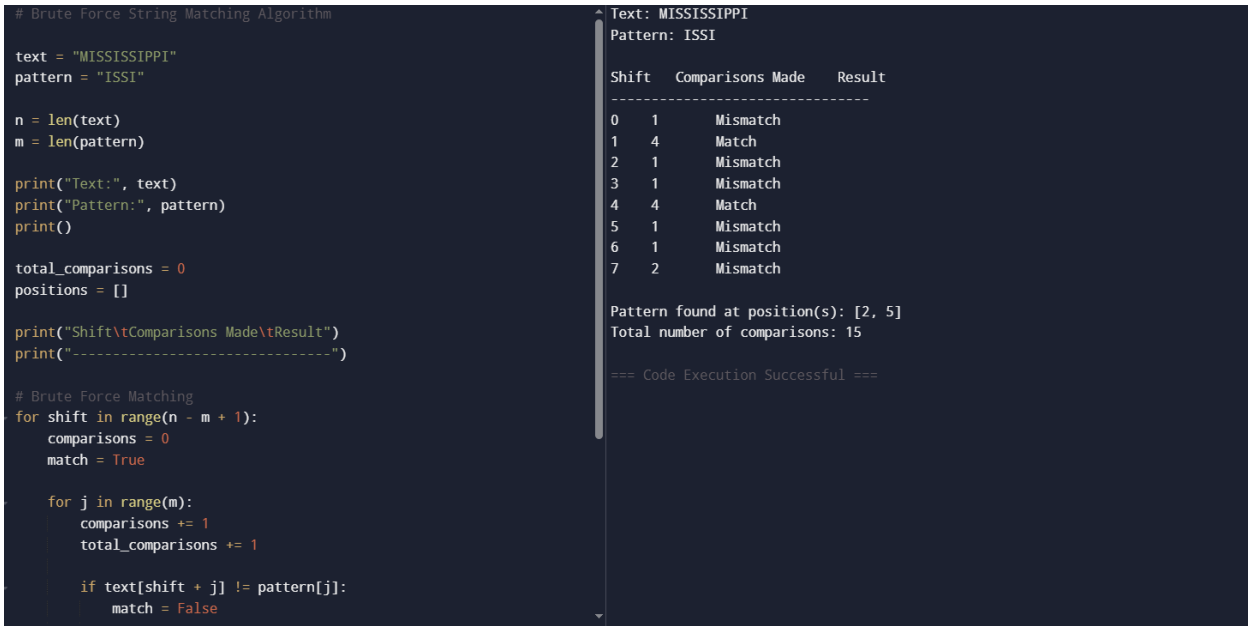
Question 3

Implement Brute Force String Matching and display the comparison table for each alignment.
Input:

- Text = "MISSISSIPPI"
- Pattern = "ISSI"

Output:

- Shift number
- Comparisons made
- Match/Mismatch result



```python
# Brute Force String Matching Algorithm

text = "MISSISSIPPI"
pattern = "ISSI"

n = len(text)
m = len(pattern)

print("Text:", text)
print("Pattern:", pattern)
print()

total_comparisons = 0
positions = []

print("Shift\tComparisons Made\tResult")
print("--------------------------------")

# Brute Force Matching
for shift in range(n - m + 1):
    comparisons = 0
    match = True

    for j in range(m):
        comparisons += 1
        total_comparisons += 1

        if text[shift + j] != pattern[j]:
            match = False
```

```
Text: MISSISSIPPI
Pattern: ISSI

Shift   Comparisons Made    Result
--------------------------------
0    1       Mismatch
1    4       Match
2    1       Mismatch
3    1       Mismatch
4    4       Match
5    1       Mismatch
6    1       Mismatch
7    2       Mismatch

Pattern found at position(s): [2, 5]
Total number of comparisons: 15

=== Code Execution Successful ===
```

Question 4

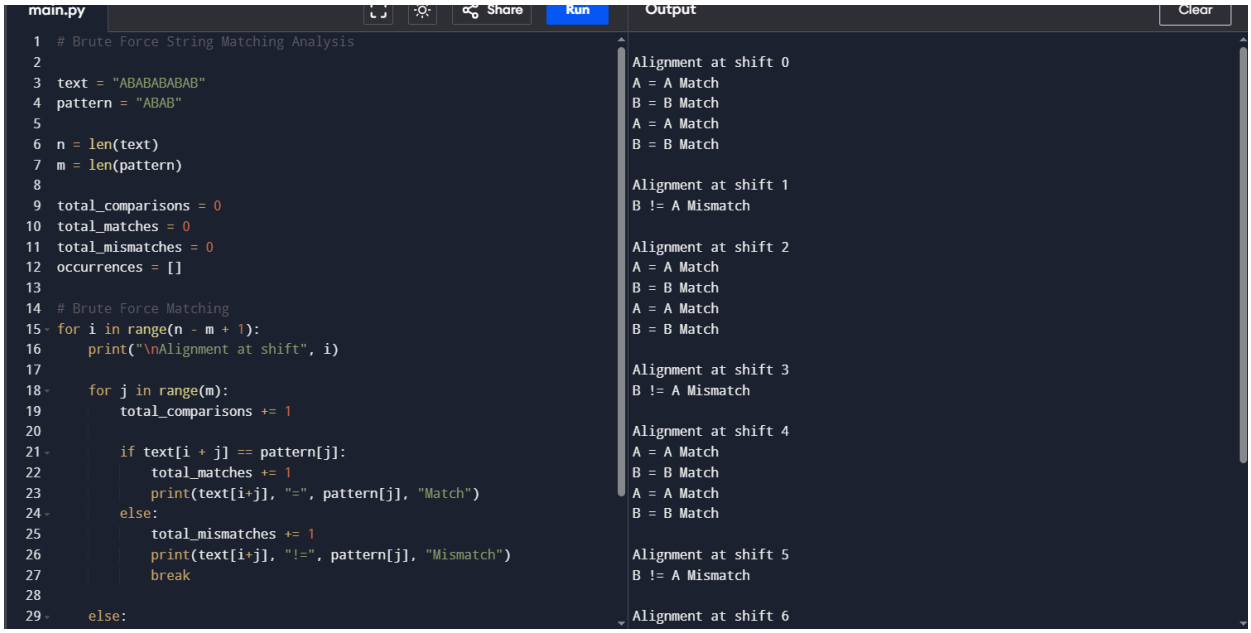
Write a program to count:

- Total character comparisons
- Total matches
- Total mismatches

during Brute Force String Matching.

Input:

- Text = "ABABABABAB"
- Pattern = "ABAB"



Question 5

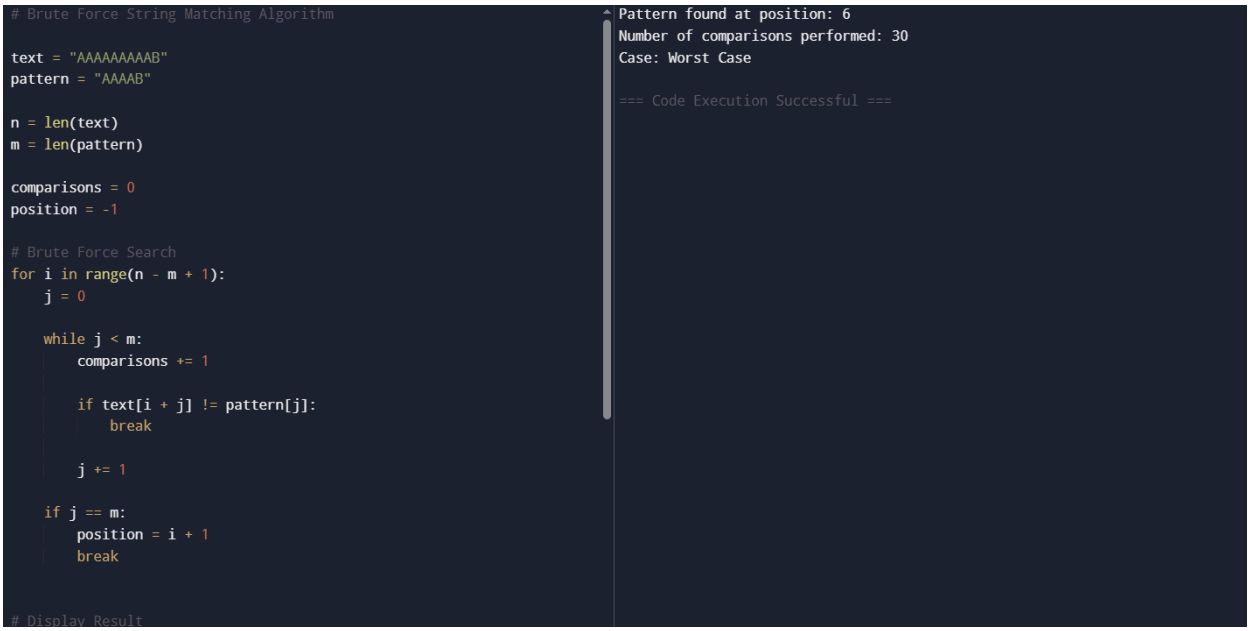
Implement Brute Force String Matching and determine whether the search is:

- Best Case
- Average Case
- Worst Case

Input:

- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.



```
# Brute Force String Matching Algorithm

text = "AAAAAAAAAB"
pattern = "AAAAB"

n = len(text)
m = len(pattern)

comparisons = 0
position = -1

# Brute Force Search
for i in range(n - m + 1):
    j = 0

    while j < m:
        comparisons += 1

        if text[i + j] != pattern[j]:
            break

        j += 1

    if j == m:
        position = i + 1
        break


# Display Result
```

```
Pattern found at position: 6
Number of comparisons performed: 30
Case: Worst Case

=== Code Execution Successful ===
```

Question 6

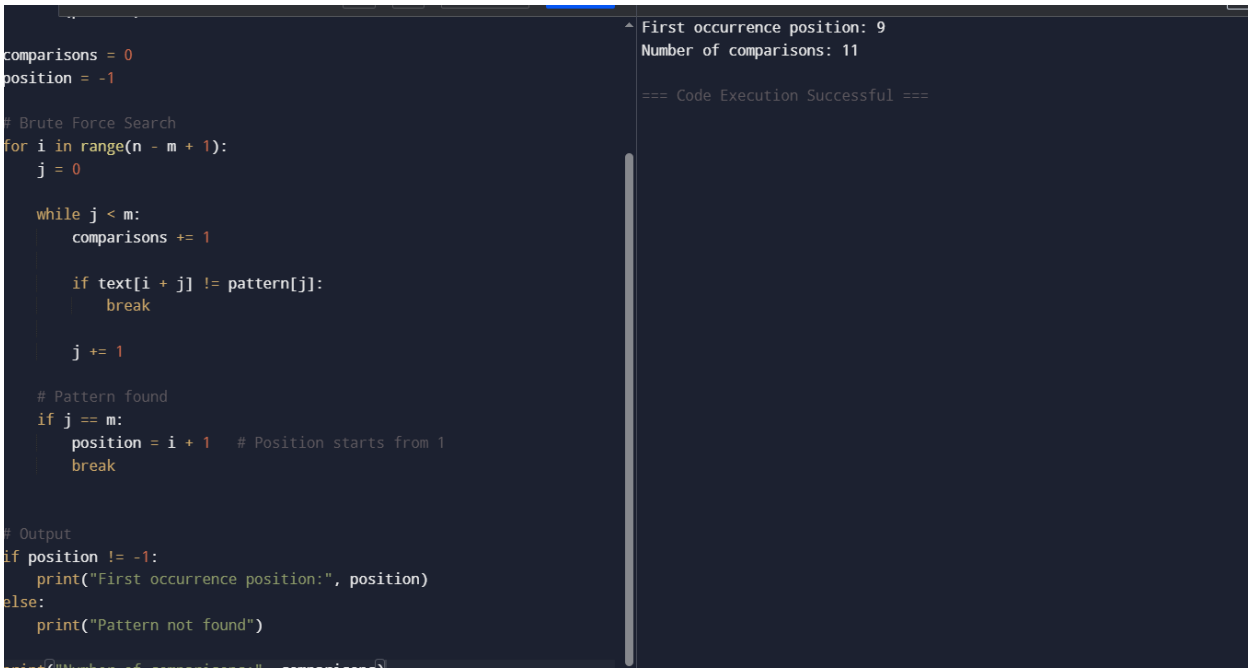
Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.

Input:

- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"

Output:

- First occurrence position
- Number of comparisons



```
comparisons = 0
position = -1

# Brute Force Search
for i in range(n - m + 1):
    j = 0

    while j < m:
        comparisons += 1

        if text[i + j] != pattern[j]:
            break

        j += 1

    # Pattern found
    if j == m:
        position = i + 1   # Position starts from 1
        break


# Output
if position != -1:
    print("First occurrence position:", position)
else:
    print("Pattern not found")
```

```
First occurrence position: 9
Number of comparisons: 11

=== Code Execution Successful ===
```

Question 7

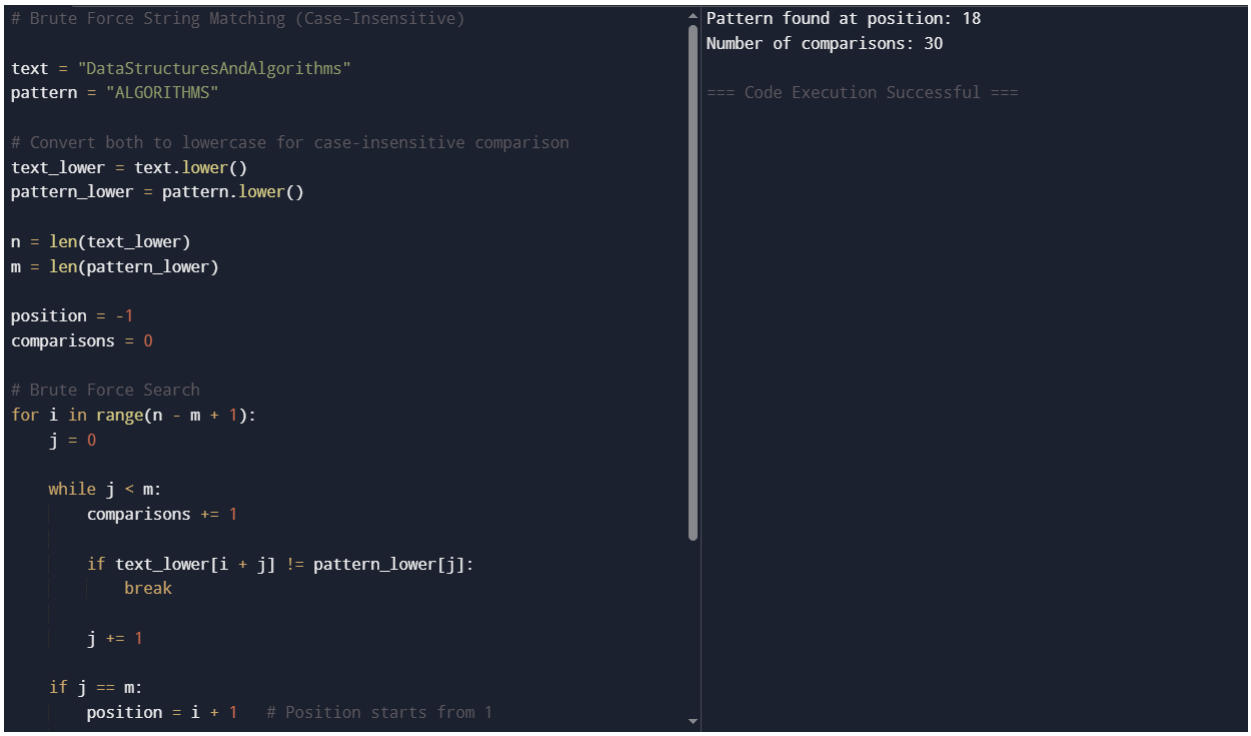
Implement Brute Force String Matching for case-insensitive searching.

Input:

- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"

Output:

- Pattern found position



```python
# Brute Force String Matching (Case-Insensitive)

text = "DataStructuresAndAlgorithms"
pattern = "ALGORITHMS"

# Convert both to lowercase for case-insensitive comparison
text_lower = text.lower()
pattern_lower = pattern.lower()

n = len(text_lower)
m = len(pattern_lower)

position = -1
comparisons = 0

# Brute Force Search
for i in range(n - m + 1):
    j = 0

    while j < m:
        comparisons += 1

        if text_lower[i + j] != pattern_lower[j]:
            break

        j += 1

    if j == m:
        position = i + 1   # Position starts from 1
```

```
Pattern found at position: 18
Number of comparisons: 30

=== Code Execution Successful ===
```

Question 8

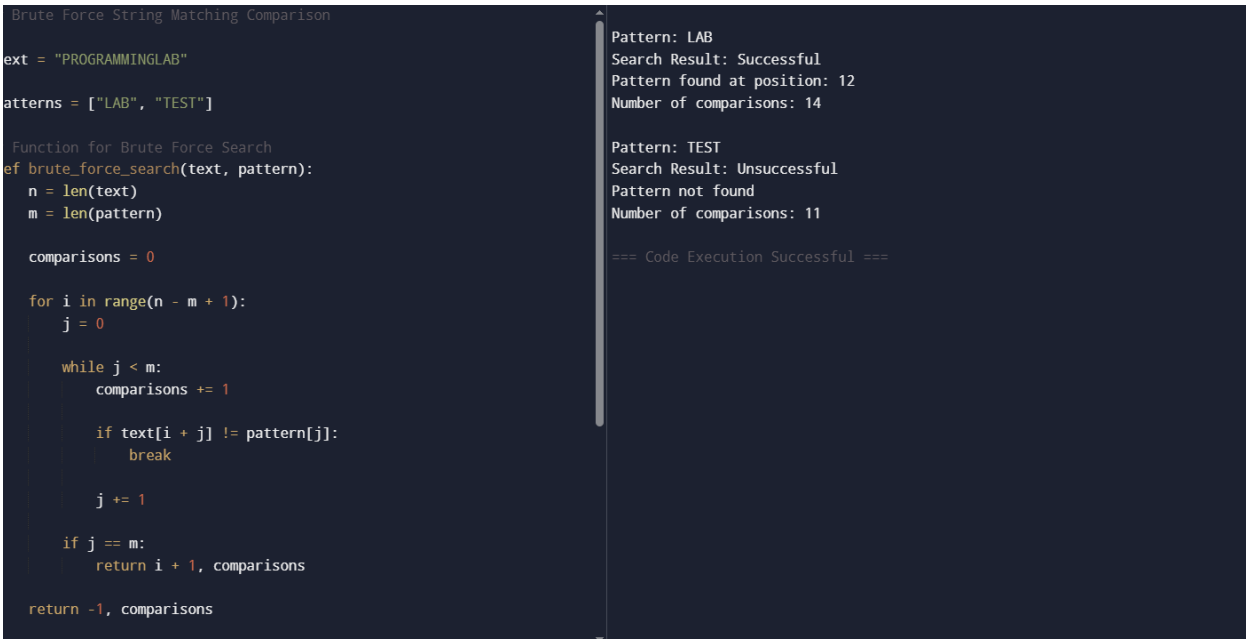
Write a program to compare the execution of Brute Force String Matching for:

1. Successful search
2. Unsuccessful search

Input:

- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"

Display the comparison count for both cases.



```
Brute Force String Matching Comparison

ext = "PROGRAMMINGLAB"

atterns = ["LAB", "TEST"]

Function for Brute Force Search
ef brute_force_search(text, pattern):
   n = len(text)
   m = len(pattern)

   comparisons = 0

   for i in range(n - m + 1):
      j = 0

      while j < m:
         comparisons += 1

         if text[i + j] != pattern[j]:
            break

         j += 1

      if j == m:
         return i + 1, comparisons

   return -1, comparisons
```

```
Pattern: LAB
Search Result: Successful
Pattern found at position: 12
Number of comparisons: 14

Pattern: TEST
Search Result: Unsuccessful
Pattern not found
Number of comparisons: 11

=== Code Execution Successful ===
```

Question 9

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:

- Text = "ABCDABCABCDA"
- Pattern = "ABCDA"

Output:

- Alignment number
- Matching result
- Pattern occurrence positions

```
1
2  text = "ABCDABCABCDA"
3  pattern = "ABCDA"
4
5  n = len(text)
6  m = len(pattern)
7
8  occurrences = []
9  alignment = 0
10
11 print("Text:", text)
12 print("Pattern:", pattern)
13 print()
14
15 print("Alignment\tPosition\tResult")
16 print("---------------------------------")
17
18 # Brute Force Matching
19 for i in range(n - m + 1):
20     alignment += 1
21     j = 0
22
23     while j < m:
24         if text[i + j] != pattern[j]:
25             break
26         j += 1
27
28     if j == m:
```

```
Text: ABCDABCABCDA
Pattern: ABCDA

Alignment   Position    Result
---------------------------------
1           1   Match
2           2   Mismatch
3           3   Mismatch
4           4   Mismatch
5           5   Mismatch
6           6   Mismatch
7           7   Mismatch
8           8   Match

Pattern occurrence positions: [1, 8]

=== Code Execution Successful ===
```

Question 10

Write a complete program for Brute Force String Matching and perform complexity analysis.

Input:

- Text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
- Pattern = "TATCTT"

Tasks:

1. Find all occurrences of the pattern.
2. Count total comparisons.
3. Display the shifting process.
4. Analyze:
    - Best-case complexity
    - Worst-case complexity
    - Space complexity

```python
# Brute Force String Matching Algorithm

text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
pattern = "TATCTT"

n = len(text)
m = len(pattern)

occurrences = []
total_comparisons = 0

print("Text:", text)
print("Pattern:", pattern)
print("\nShifting Process:")
print("---------------------------------")

# Brute Force Matching
for i in range(n - m + 1):
    comparisons = 0
    j = 0

    print("Shift", i, ":", text[i:i+m], end=" --> ")

    while j < m:
        comparisons += 1
        total_comparisons += 1

        if text[i + j] != pattern[j]:
```

```
Text: TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT
Pattern: TATCTT

Shifting Process:
---------------------------------
Shift 0 : TTATAG --> Mismatch
Comparisons in this shift: 2
Shift 1 : TATAGA --> Mismatch
Comparisons in this shift: 4
Shift 2 : ATAGAT --> Mismatch
Comparisons in this shift: 1
Shift 3 : TAGATC --> Mismatch
Comparisons in this shift: 3
Shift 4 : AGATCT --> Mismatch
Comparisons in this shift: 1
Shift 5 : GATCTC --> Mismatch
Comparisons in this shift: 1
Shift 6 : ATCTCG --> Mismatch
Comparisons in this shift: 1
Shift 7 : TCTCGT --> Mismatch
Comparisons in this shift: 2
Shift 8 : CTCGTA --> Mismatch
Comparisons in this shift: 1
Shift 9 : TCGTAT --> Mismatch
Comparisons in this shift: 2
Shift 10 : CGTATT --> Mismatch
Comparisons in this shift: 1
Shift 11 : GTATTC --> Mismatch
```